UNIT-IV: Transport Layer

Process to Process CommunicationUser Datagram Protocol (UDP), Transmission Control Protocol (TCP)SCTP Congestion Control,Quality of Service
QoS improving techniques: Leaky Bucket and Token Bucket algorithm.

Transport Layer

The **Transport Layer** ensures **process-to-process delivery** of complete messages, allowing communication between specific applications (processes) on different hosts.

Key Concepts

1. **Process-to-Process Communication**
    - A **process** is an application running on a host.
    - Delivery is from a specific process on one computer to a process on another, using **port numbers** in the TCP/IP suite (16-bit identifiers).
2. **Connection Types**
    - **Connectionless (UDP)**: Each segment is independent, with no flow or error control.
    - **Connection-Oriented (TCP, SCTP)**: Establishes a connection before data transfer, ensuring reliable delivery.
3. **Socket Address**
    - A combination of **IP address** and **port number** uniquely identifies a process on a host.

Example Network

In a network (A → B → C → D → E):

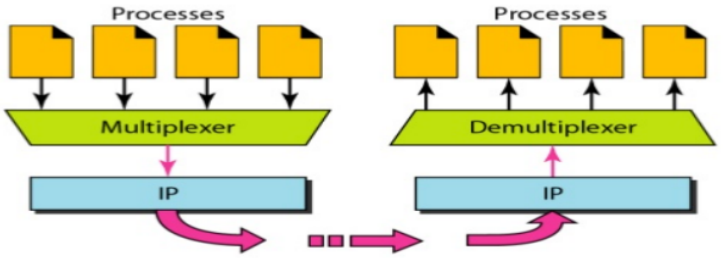
- **Source**: A | **Destination**: E | **Nodes**: A, B, C, D, E
- **Data Link Layer**: Hop-to-Hop (MAC addresses)
- **Network Layer**: Host-to-Host (IP addresses)
- **Transport Layer**: End-to-End (Port numbers)

Responsibilities of the Transport Layer

1. **Process-to-Process Delivery**
    - Uses **port numbers** to deliver data to the correct process on a host.
    - **IANA Port Ranges**:
        - **Well-Known Ports (0–1023)**: Assigned by IANA (e.g., HTTP on 80).
        - **Registered Ports (1024–49151)**: Can be registered to avoid duplication.
        - **Dynamic Ports (49152–65535)**: Used by any process dynamically.
2. **End-to-End Connection**
    - **TCP**: Reliable, connection-oriented, uses a handshake for setup and ensures message delivery.
    - **UDP**: Unreliable, connectionless, used for speed-critical applications like streaming.
3. **Multiplexing and Demultiplexing**

- **Multiplexing**: Combines data from multiple processes at the sender into a single packet.
- **Demultiplexing**: Splits received data at the receiver, directing it to the correct processes.

Figure 23.6 Multiplexing and demultiplexing

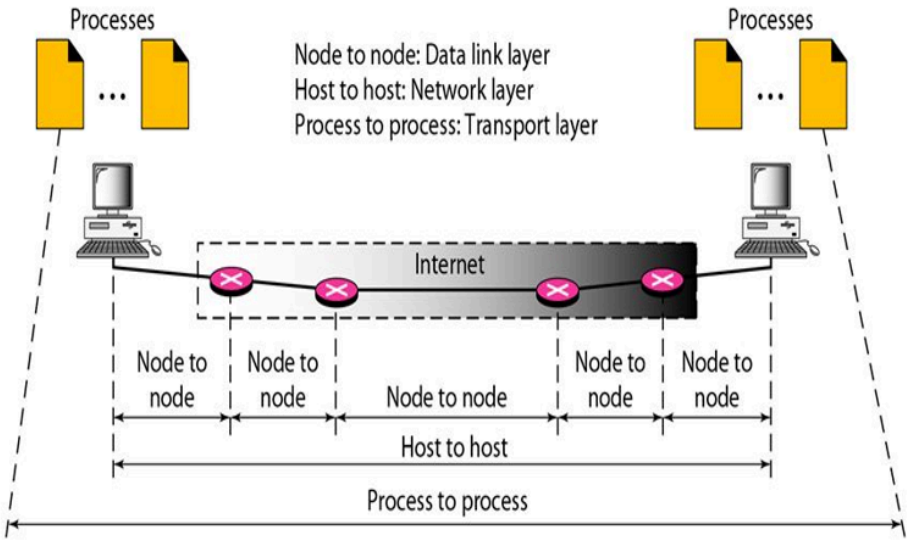


4. **Congestion Control**
   - Manages network congestion using:
     - **Open-loop control**: Prevents congestion before it happens.
     - **Closed-loop control**: Manages congestion after it occurs, reducing packet loss.
5. **Data Integrity and Error Control**
   - Uses **checksums**, **ACK (Acknowledgment)**, and **NACK (Negative Acknowledgment)** to detect and correct errors.
6. **Flow Control**
   - Ensures sender and receiver operate at compatible speeds using the **sliding window protocol**, where the receiver specifies how much data it can handle.

Store and Forward is a data transfer method where the receiving device stores the entire data packet, checks for errors, and then forwards it to the next device or destination. This process ensures data integrity but introduces some delay, as the device must receive and process the entire packet before sending it on. Examples include email servers and network routers.

Process-to-process communication is the core responsibility of the Transport Layer in the OSI and TCP/IP models. It ensures that data is delivered from a specific process (application) on one host to a specific process on another host.

It ensures data is delivered accurately and reliably between specific processes on different hosts using socket addresses, port numbers, and appropriate protocols like TCP or UDP.



Types of data deliveries

Key Components:

1. **Processes**
    - A process is an application or program running on a device (host).
    - Examples: A web browser (client process) requesting data from a web server (server process).
2. **Socket Address**
    - To uniquely identify processes, the Transport Layer uses a **socket address**, which is a combination of:
        - **IP Address**: Identifies the host.
        - **Port Number**: Identifies the specific process on the host.
3. **Port Numbers**
    - Port numbers (16-bit) allow multiple processes to run simultaneously on the same host.
    - Examples:
        - HTTP (port 80), HTTPS (port 443), FTP (port 21).
4. **Delivery Mechanism**
    - **Source-to-Destination**: Transports data from one host to another.
    - **End-to-End**: Ensures delivery to the correct processes on each host.

Example:

Scenario: A web browser on Host A requests a web page from a web server on Host B.

- **Host A (Client Process)**: IP address 192.168.1.2, port 49152.
- **Host B (Server Process)**: IP address 10.0.0.5, port 80 (HTTP).
- **Socket Pair**: (192.168.1.2:49152, 10.0.0.5:80).
- The transport layer ensures the data sent by the web server is delivered directly to the web browser on Host A.

Connection-Oriented (CO) and Connectionless (CL) Protocols

Connection-Oriented (CO) Protocols

1. **Connection Establishment**: A dedicated connection is established before data transfer.
2. **Reliable Data Transfer**: Ensures data is transferred reliably through:
    - **Handshaking** (setup and teardown).
    - **Sequencing** (ordering of packets).
    - **Acknowledgments (ACKs)**.
    - **Flow control** to prevent congestion.
    - **Error detection and correction**.
3. **Examples**:
    - **TCP (Transmission Control Protocol)**
    - **ATM (Asynchronous Transfer Mode)**

Connectionless (CL) Protocols

1. **No Connection Establishment**: Data is sent without establishing a connection first.
2. **No Guarantees**:
    - No guarantee of **delivery**.
    - No guarantee of **ordering**.
    - No guarantee of **error-free transmission**.
3. **Faster Transmission**: Due to reduced overhead.
4. **Examples**:
    - **UDP (User Datagram Protocol)**
    - **IP (Internet Protocol)**
    - **ICMP (Internet Control Message Protocol)**

Aspect	Connection-Oriented (CO)	Connectionless (CL)
Connection Establishment	Yes, a dedicated connection is established.	No, no connection is established.
Reliability	High (ensures delivery, error correction, sequencing).	Low (no guarantees on delivery or order).
Overhead	Higher (due to handshaking, ACKs, error handling).	Lower (minimal overhead).
Speed	Slower (due to connection setup and error handling).	Faster (due to no connection setup).
Error Handling	Built-in error handling (retransmissions, acknowledgments).	None (errors must be handled by the application).
Sequencing	Guaranteed (data packets arrive in order).	Not guaranteed (packets may arrive out of order).

Key Differences Between Connection-Oriented and Connectionless Protocols

When to Use Each

Connection-Oriented (CO)

- **File Transfers** (e.g., FTP): Ensures that data is reliably transferred.
- **Email**: Reliable and guaranteed delivery of messages.
- **Remote Access** (e.g., SSH): Requires a stable and reliable connection.
- **Applications requiring guaranteed delivery**: For example, online banking.

Connectionless (CL)

- **Real-time applications** (e.g., Voice, Video): Speed and low latency are more important than reliability.
- **Online Gaming**: Some packet loss is acceptable, but speed is crucial.
- **Streaming Media** (e.g., live video streaming): Loss of a few packets does not disrupt the overall experience.
- **Applications tolerating some packet loss**: For instance, DNS queries where the occasional lost packet is not critical.

Real-World Analogies

- **Connection-Oriented**: **Making a Phone Call** – A connection is established before communication begins, and the connection is maintained until the conversation ends.
- **Connectionless**: **Sending a Postcard** – You send a message with no guarantee that it will arrive at the destination or that it will be delivered in order.

Transport Layer Protocols

• TCP(Transmission Control Protocol)

• UDP(User Datagram Protocol)

• SCTP (Stream Control Transmission Protocol):**SCTP (Stream Control Transmission Protocol)** congestion control is designed to manage the flow of data over a network to avoid congestion while ensuring efficient and reliable communication. SCTP uses a combination of techniques, similar to those in TCP, but with some unique features to handle congestion effectively, especially in multi-homing environments (i.e., where multiple network paths can be used simultaneously).

SCTP (Stream Control Transmission Protocol):

- A transport layer protocol designed to transport multiple streams of data between two endpoints, while maintaining message boundaries.
- It provides features like **multi-homing**, which allows multiple IP addresses per endpoint for fault tolerance, and **message-oriented communication**.

- SCTP is used in applications requiring reliable data transfer, like **telecommunications** and **signaling** (e.g., for **SS7** signaling).
- Unlike TCP, SCTP supports **multi-streaming** (multiple independent streams within one connection) to avoid head-of-line blocking.

TCP (Transmission Control Protocol) TCP is a reliable, connection-oriented protocol that provides process-to-process communication and ensures the accurate delivery of data in the correct order.

TCP Services

1. **Process-to-Process Communication**
   - Uses **port numbers** to identify specific processes on each host.
2. **Reliable Communication**
   - Implements **flow control** and **error control** mechanisms.
   - Uses **acknowledgment (ACK)** and **negative acknowledgment (NACK)** to ensure data delivery.
3. **Data Ordering**
   - Maintains the **order of data** by numbering each segment, ensuring that data is reassembled correctly at the destination.
4. **Connection-Oriented**
   - Establishes a connection before data transfer (using a handshake) and terminates it after transfer is complete.
5. **Full Duplex**
   - Supports **bidirectional data transfer**, allowing simultaneous data exchange in both directions.
6. **Stream-Oriented**
   - Transfers data as a continuous **stream of bytes**, enabling efficient communication through a virtual circuit.

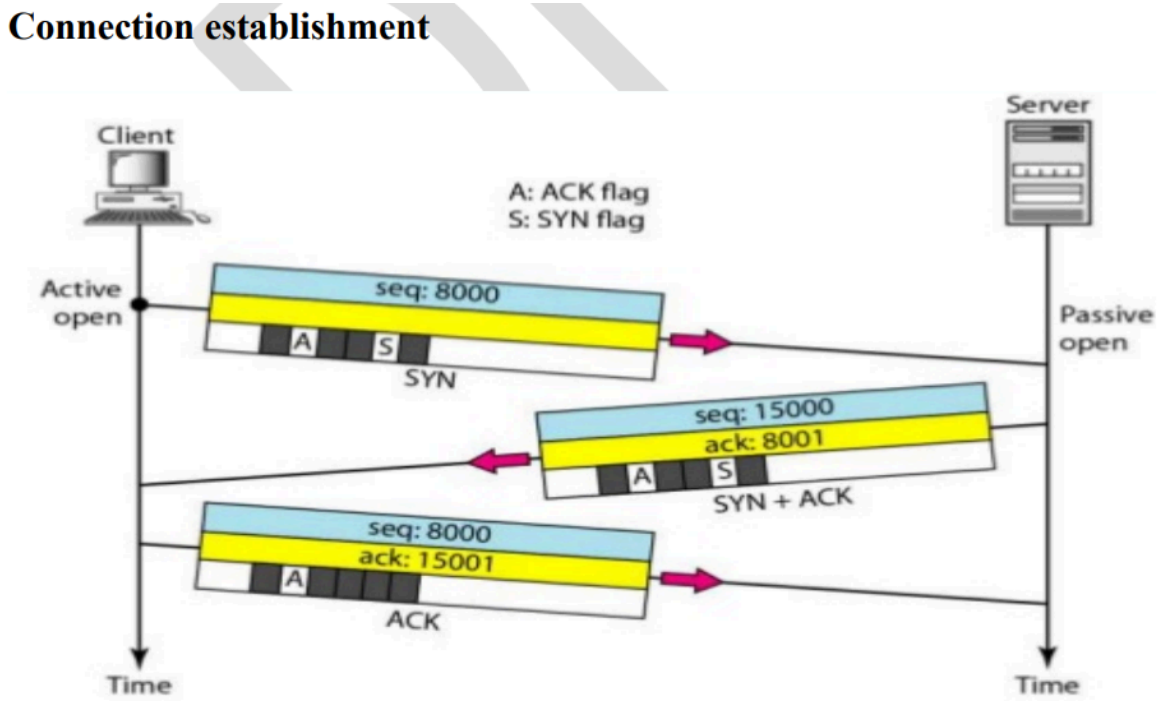
Key Features in Detail

1. **Flow Control (Sliding Window)**
   - Prevents data loss by ensuring that the sender does not overwhelm the receiver. The receiver specifies how much data it can handle using the **window size**.
2. **Error Control (Checksum & ACK)**
   - **Checksum**: Ensures integrity by detecting errors in transmitted data.
   - **ACK/NACK**: Confirms successful or failed delivery of data.
3. **Three-Way Handshake (Connection Setup)**
   - **SYN**: Sender requests a connection.
   - **SYN-ACK**: Receiver acknowledges and responds.
   - **ACK**: Sender confirms, establishing the connection.
4. **Graceful Termination (Connection Teardown)**
   - **FIN/ACK**: Both parties agree to close the connection using **FIN** and **ACK** flags.

5. **Congestion Control**:
   Manages network congestion through mechanisms like **slow start**, **congestion avoidance**, and **fast recovery**.
6. **Multiplexing/Demultiplexing**:
   Allows multiple processes to share the same network connection by differentiating them with port numbers.

A **TCP connection** involves three primary phases: **Connection Establishment**, **Data Transfer**, and **Connection Termination**.

1. **Connection Establishment (Three-Way Handshake)**To establish a reliable TCP connection, the client and server go through a process called **three-way handshake**. This is how it works:



Three-Way Handshake Process:

1. **Step 1 (SYN):**
   The client sends a **SYN** (synchronize) segment to the server to initiate a connection. This segment contains the sequence number that the client will use for the data transfer.
2. **Step 2 (SYN + ACK):**
   The server responds with a **SYN + ACK** segment, indicating that it is ready to establish a connection. The **SYN** flag indicates that the server is willing to communicate, and the **ACK** flag acknowledges the client's initial request.
3. **Step 3 (ACK):**
   The client sends an **ACK** (acknowledgment) segment to the server, confirming that the connection has been successfully established in both directions.

After this handshake, both the client and server are synchronized and can start exchanging data.

Simultaneous Open: In rare cases, both the client and server might initiate the connection at the same time, leading to both sending **SYN + ACK** segments simultaneously. This results in a connection being established in both directions.

SYN Flooding Attack: This type of **Denial of Service (DoS) attack** involves a malicious actor sending numerous fake **SYN** requests to a server, causing it to allocate resources unnecessarily, eventually leading to a server crash or resource exhaustion.

2. Data Transfer

Once the connection is established, data transfer takes place in full-duplex mode, meaning both the client and server can send and receive data simultaneously.

Data Transmission:

- **Acknowledgment**: Both data and acknowledgment are transmitted together (piggybacking), increasing efficiency.
- **Pushing Data**: The sending TCP stores data in a buffer and controls the segment size. The receiving TCP buffers incoming data until it's ready for processing by the application.
- **Urgent Data**: If the application needs to send urgent data out of order, TCP provides a mechanism to prioritize these urgent bytes and ensure their immediate delivery.

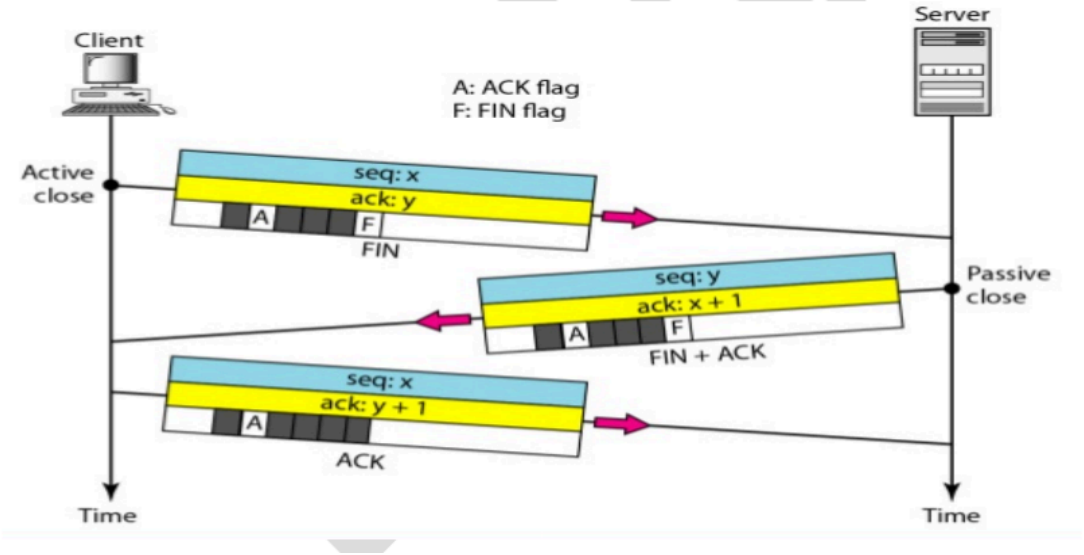
3. Connection Termination

After data transfer is complete, either the client or server can initiate the termination of the connection. The most common termination process is the **four-way handshake**

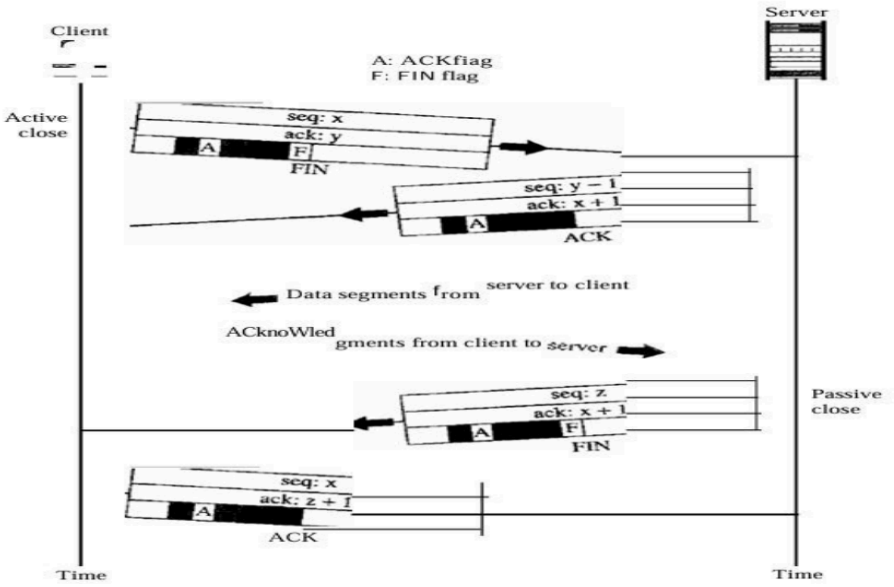
Four-Way Handshake Termination:

1. **FIN (Finish)**
    - **Client** sends a **FIN** packet to the **Server**, indicating that it has finished sending data and wants to close the connection.
2. **ACK (Acknowledge)**
    - **Server** acknowledges the **FIN** packet with an **ACK**. The server is still able to send data to the client at this point.
3. **FIN (Finish)**
    - **Server** sends its own **FIN** packet to the **Client**, indicating that it has finished sending data.
4. **ACK (Acknowledge)**
    - **Client** sends an **ACK** to the **Server**, acknowledging the server's **FIN**. Once this is done, the connection is fully closed.

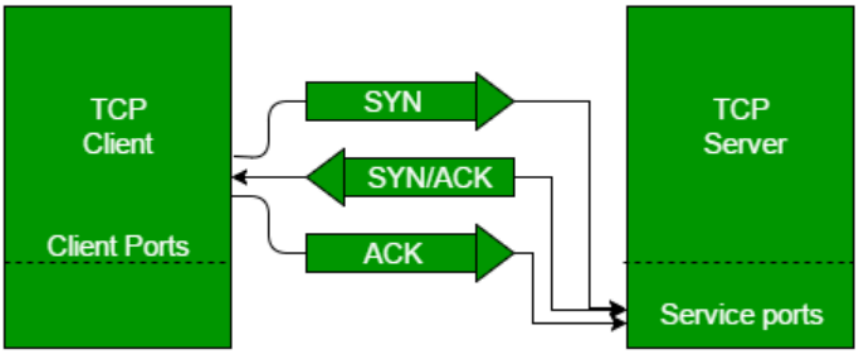
Connection termination



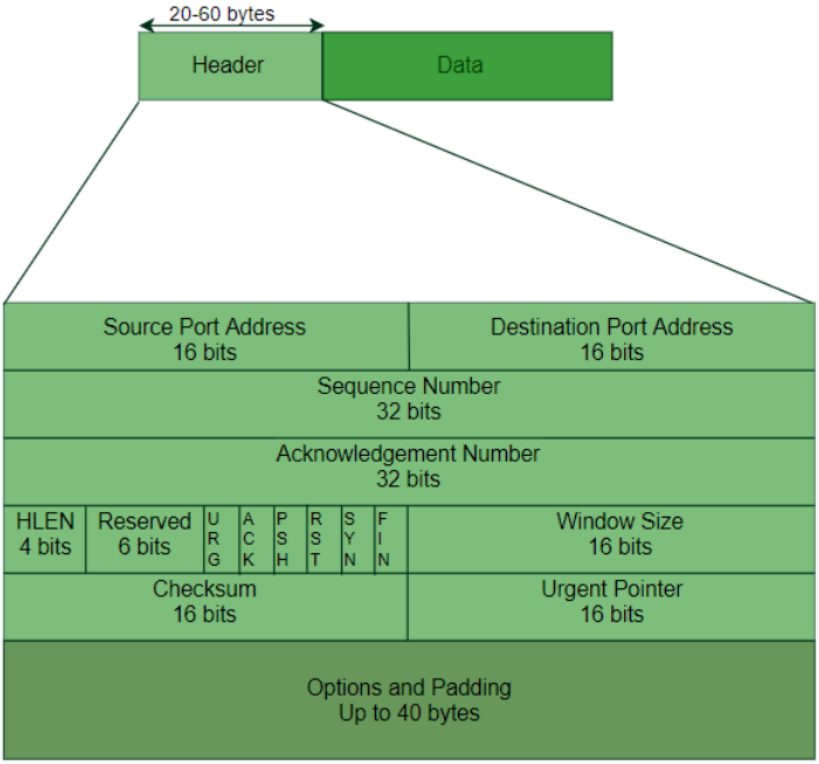
Half-Close: In some scenarios, one side of the connection (typically the client) may stop sending data but continue to receive it. This is called a **half-close**, and it's useful when the client no longer needs to send data but still requires responses from the server.



Flow Control in TCP TCP uses a **sliding window** mechanism to handle flow control, ensuring that the sender doesn't overwhelm the receiver with too much data at once. This sliding window protocol works as a buffer and can be considered a combination of **Go-Back-N** and **Selective Repeat** strategies, providing a balance between performance and reliability.


TCP
Client
Client Ports
SYN
SYN/ACK
ACK
TCP
Server
Service ports

TCP SEGMENT


20-60 bytes
Header
Data
Source Port Address
16 bits
Destination Port Address
16 bits
Sequence Number
32 bits
Acknowledgement Number
32 bits
HLEN
4 bits
Reserved
6 bits
URG
ACK
PSH
RST
SYN
FIN
Window Size
16 bits
Checksum
16 bits
Urgent Pointer
16 bits
Options and Padding
Up to 40 bytes

TCP Header Format

The TCP header consists of several fields used to control the transmission of data. It is a fixed-size header, typically 20 bytes, but can extend with options. Here's the format:

Field	Size (bits)	Description
Source Port	16	The port number of the sender.
Destination Port	16	The port number of the receiver.
Sequence Number	32	Identifies the byte stream sent by the sender.
Acknowledgment Number	32	The next expected byte to be received, used for flow control.
Data Offset (Header Length)	4	The length of the TCP header (in 32-bit words).
Reserved	3	Reserved for future use, should be zero.
Flags	9	Control flags (URG, ACK, PSH, RST, SYN, FIN).
Window Size	16	Specifies the size of the receiver's buffer space.
Checksum	16	Error-checking value for the header and data.
Urgent Pointer	16	Used if the URG flag is set, indicating urgent data.
Options (if any)	Variable	Extra options, if any, following the header.
Padding	Variable	Us .o align the header to a multiple of 32 bits.

Control. This field defines 6 different control bits or flags

Flag	*Description*
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

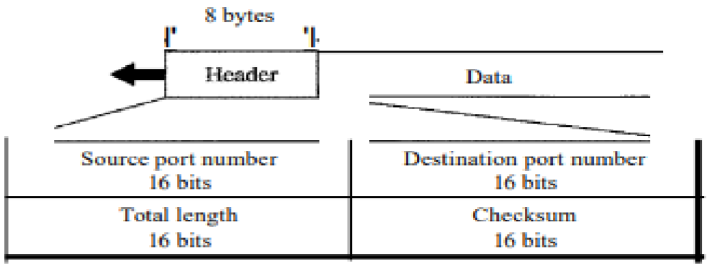
User Datagram Protocol (UDP)

UDP is a **connectionless**, **unreliable** transport layer protocol in the TCP/IP model. It is simple and lightweight, providing minimal overhead, making it suitable for applications where speed is more critical than reliability.

Key Features of UDP

- **Connectionless Service**:
  Each UDP datagram is independent, meaning there is no connection establishment or termination. UDP does not guarantee message delivery, order, or error recovery.

- **Unreliable**:
  There is no flow control or error control in UDP. If a message is lost or corrupted, it is not retransmitted, and no acknowledgment is sent to the sender.
- **Low Overhead**:
  UDP's simple structure requires less processing than protocols like TCP, making it ideal for applications where speed is essential and occasional packet loss is acceptable.
- **Simple and Fast**:
  Since UDP avoids the overhead of connection management, it is faster than TCP, suitable for time-sensitive applications like streaming.



UDP Header Format

The UDP header is simpler and smaller than the TCP header, and it is always 8 bytes. Here's the format:

Field	Size (bits)	Description
Source Port	16	The port number of the sender.
Destination Port	16	The port number of the receiver.
Length	16	The length of the UDP header and data in bytes.
Checksum	16	Error-checking value for the header and data (optional in IPv4 but mandatory in IPv6).

In **UDP**, the focus is on simplicity and low overhead, which makes it faster but less reliable than TCP.

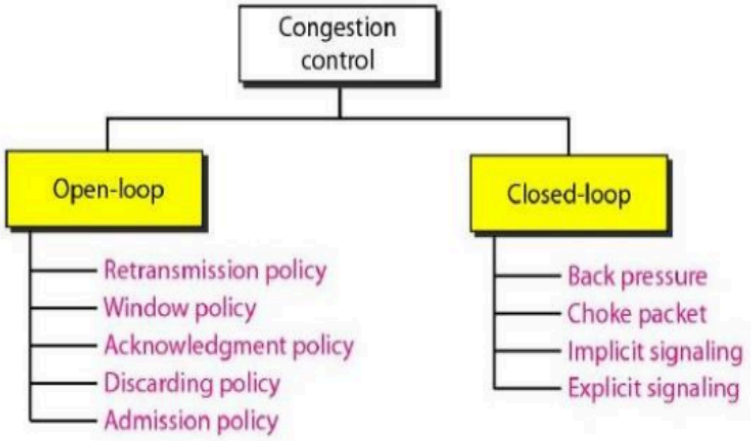
UDP Operation

- **No Connection Establishment**:
  UDP does not set up or terminate a connection. Each datagram is independent.
- **No Flow Control**:
  The sender does not regulate data flow, meaning the receiver may be overwhelmed with data if it cannot keep up.
- **Error Control**:
  The only error control in UDP is the checksum, which detects errors in the header

and data. If errors are found, the datagram is discarded. There is no retransmission of lost or corrupted data.

- **Encapsulation and Decapsulation**:
  UDP encapsulates application data into a UDP datagram and adds the necessary header information. This datagram is then encapsulated into an IP packet for transmission across the network. Upon receipt, the datagram is decapsulated by UDP at the destination.

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-oriented (requires connection setup before data transfer)	Connectionless (no connection establishment)
Reliability	Reliable (guarantees delivery, order, and error correction)	Unreliable (no guarantee of delivery, order, or error handling)
Flow Control	Yes (uses mechanisms like sliding window)	No flow control (sender can overwhelm the receiver)
Error Control	Yes (uses checksums, acknowledgment, and retransmissions)	Limited error control (only checksums; no retransmission)
Data Integrity	Ensures that data is received in the correct order without errors	No assurance of data integrity or order
Data Transmission	Stream-oriented (data is sent as a continuous stream of bytes)	Message-oriented (data is sent in discrete chunks or datagrams)
Header Size	Larger (20-60 bytes)	Smaller (8 bytes)
Congestion Control	Yes (avoids congestion by adjusting data flow)	No congestion control
Speed	Slower (due to overhead from connection setup, error recovery, and flow control)	Faster (minimal overhead, no connection or error handling)
Usage	Suitable for applications where reliability is crucial (e.g., web browsing, email, file transfers)	Suitable for applications where speed is crucial and reliability is less important (e.g., live streaming, VoIP, gaming)
Segment Structure	Includes sequence numbers, acknowledgments, and control flags	Includes source and destination ports, checksum, and length
Overhead	High (due to connection setup, flow control, and reliability mechanisms)	Low (minimal overhead, no connection or reliability mechanisms)



Congestion and Congestion Control

Congestion occurs in a packet-switched network when the data load exceeds the network's capacity, leading to packet loss, delays, or network failures. **Congestion control** refers to the techniques used to either **prevent congestion** before it happens or **mitigate it** once it occurs. The goal is to maintain efficient data transfer and prevent network overload.

1. Open-Loop Congestion Control (Prevention)

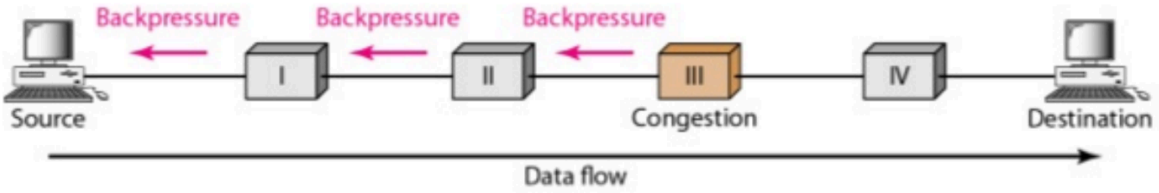
Open-loop congestion control aims to prevent congestion **before it occurs**. Policies are applied either at the source or destination to manage traffic.

- **Retransmission Policy**:
  - Retransmissions can increase congestion by reintroducing lost or corrupted packets. Proper design of retransmission policies, such as setting optimal retransmission timers, is crucial for avoiding additional congestion.
- **Window Policy**:
  - The sender's window size (e.g., Selective Repeat) impacts congestion. A larger window size may lead to higher traffic, while a smaller window size can help reduce congestion.
- **Acknowledgment Policy**:
  - The receiver's acknowledgment policy can help control congestion. For example, the receiver may acknowledge only a certain number of packets at once or use timers to send acknowledgments, which can help manage the flow of traffic.
- **Discarding Policy**:
  - Routers use discarding policies to drop packets when necessary. Good discarding policies ensure that dropped packets do not harm the transmission integrity or contribute to congestion unnecessarily.
- **Admission Policy**:
  - In virtual-circuit networks, routers may use an admission policy to deny new virtual circuit connections if congestion is detected or expected. This helps prevent network overload.

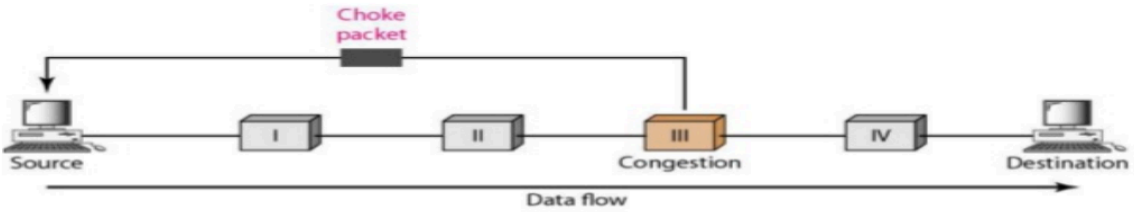
2. Closed-Loop Congestion Control (Removal)

Closed-loop congestion control focuses on alleviating congestion **after it happens** by adjusting the flow of data in response to congestion signals.

- **Backpressure**:
  - In backpressure, a congested node stops receiving data from its upstream nodes, propagating congestion signals upstream. This process can lead to congestion in upstream nodes and ultimately to the source. Backpressure is applicable only in virtual-circuit networks.



- **Choke Packet**:
  - A choke packet is a signal sent by a congested node to the source to inform it of congestion. This signal is sent directly to the source, which can then adjust its transmission rate.



- **Implicit Signaling**:
  - Implicit signaling occurs when the source infers that congestion is present based on symptoms like delayed acknowledgments or no acknowledgments. There's no direct communication from the congested node to the source.
- **Explicit Signaling**:
  - Explicit signaling involves the congested node sending a direct signal to the source or destination. It can be backward or forward signaling:
    - **Backward Signaling**: A congestion warning bit is set in packets moving against the flow of data, signaling the source to slow down transmission.
    - **Forward Signaling**: A congestion warning bit is set in packets moving toward the destination, signaling the receiver to slow down its acknowledgments.

3. Congestion Control in Specific Protocols

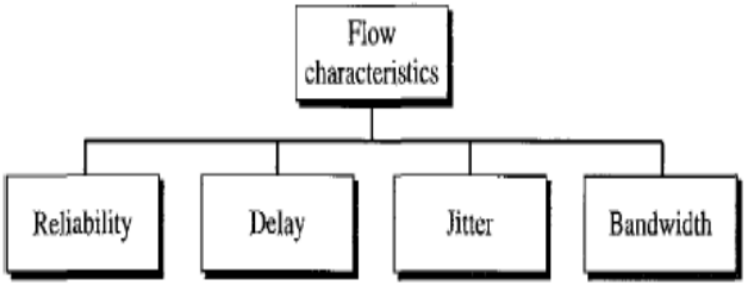
- **TCP (Transmission Control Protocol)**:TCP uses a combination of slow-start, congestion avoidance, and fast retransmit techniques to control congestion. It adjusts

the size of its congestion window based on feedback from the network and uses an additive increase/multiplicative decrease strategy for flow control.

- **SCTP (Stream Control Transmission Protocol)**:SCTP uses similar congestion control methods as TCP but also accounts for multi-homing, where data can be sent over multiple network paths. This allows for more flexibility and fault tolerance in congested environments.

Quality of Service (QoS)

Quality of Service (QoS) refers to the performance levels of network services, which are especially crucial for applications with specific requirements for reliability, delay, jitter, and bandwidth. Different applications have different QoS needs, and ensuring these needs are met involves using various techniques to optimize the flow of data across the network.



Key QoS Parameters:

1. **Reliability**:
    - Reliability ensures that data is transmitted correctly and without loss. Packet loss requires retransmission, which impacts performance. Some applications, like email and file transfers, require higher reliability, while others, such as telephony or audio conferencing, may tolerate some data loss.
2. **Delay**:
    - Delay refers to the time taken for data to travel from the source to the destination. For real-time applications like telephony, video conferencing, and remote login, low delay is crucial, whereas for file transfers and emails, some delay is acceptable.
3. **Jitter**:
    - Jitter is the variation in delay for packets in the same flow. For instance, if packets arrive with irregular delays, it can affect applications like video and audio conferencing, where consistent packet timing is critical. Applications like email are less sensitive to jitter.
4. **Bandwidth**:
    - Different applications require varying amounts of bandwidth. For example, video conferencing needs high bandwidth to handle large video and audio streams, while applications like email can function with much lower bandwidth.

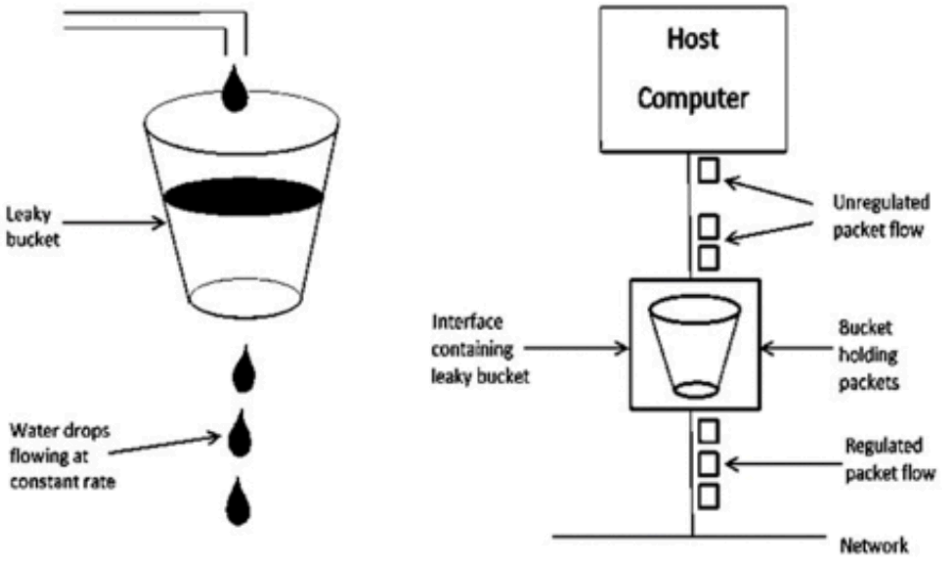
Techniques to Improve QoS

Leaky Bucket Algorithm:

The **Leaky Bucket** algorithm works by controlling the rate of traffic flowing from the source into the network, even when there are bursty inputs. Here's how it works:

- **Concept**: The leaky bucket can be visualized as a bucket with a hole at the bottom. Water (representing data packets) enters the bucket at an unpredictable rate, and it leaks out of the bucket at a constant rate as long as the bucket has water in it. The rate at which the water leaks out of the bucket is fixed and does not depend on the input rate.
- **Network Equivalent**: In networking, the leaky bucket algorithm smooths out traffic by storing bursts of data in the bucket and then releasing it at a constant rate. The goal is to ensure that the data does not exceed a certain rate, such as a network's committed bandwidth.
- **Example**:
    - Suppose a network has committed bandwidth of **3 Mbps** for a host.
    - The host sends a burst of data at **12 Mbps** for **2 seconds** (24 Mbits of data), followed by **5 seconds** of silence, and then sends data at **2 Mbps** for **3 seconds** (6 Mbits of data).
    - Without the leaky bucket, the network could experience congestion due to the burst at the beginning.
    - With the leaky bucket, the network smooths out the burst and ensures that the traffic is sent at **3 Mbps** for the entire **10 seconds** (thus, 30 Mbits of data are sent out at a consistent rate).

Advantage: The leaky bucket algorithm helps prevent congestion by smoothing out bursty traffic and ensuring a constant output rate.
Disadvantage: It doesn't account for idle time or allow for any burstiness in the traffic, making it less flexible than other approaches.



Token Bucket Algorithm:

The **Token Bucket** algorithm addresses some limitations of the leaky bucket algorithm by allowing bursts of traffic while still enforcing an average rate of transmission. Here's how it works:

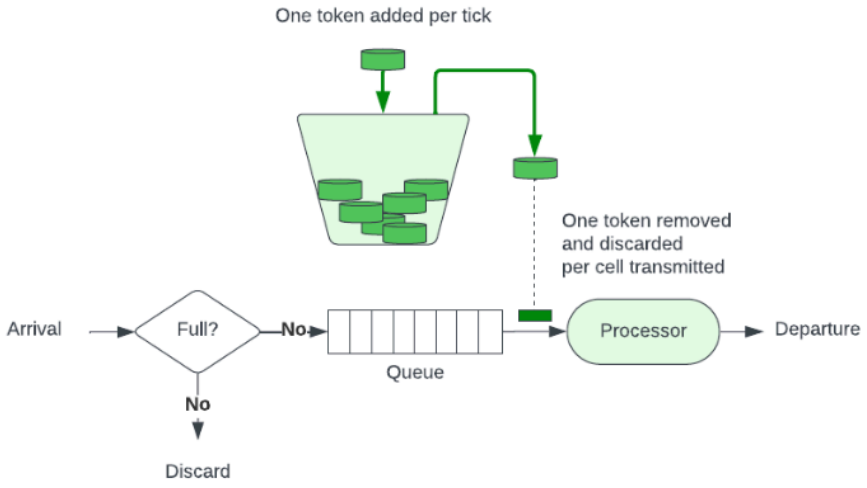
- **Concept**: The token bucket has tokens, which are generated at a regular interval (clock ticks). Each token represents permission to send a certain amount of data. The bucket collects tokens when the sender is idle, and when data is ready to be sent, it consumes tokens from the bucket. If there are enough tokens, the data can be sent at a higher rate (burst), but if the tokens run out, the sending must wait.
- **Network Equivalent**:
  - For each tick of the clock, the system sends **n tokens** to the bucket.
  - When the host has data to send, it consumes one token for each byte of data sent.
  - If the host is idle, it accumulates tokens over time. This allows the host to send bursts of data if enough tokens have been accumulated.
- **Example**:
  - Suppose the bucket receives **100 tokens per tick**. If the host is idle for **100 ticks**, it accumulates **10,000 tokens**.
  - The host can now send **10,000 bytes** of data in one tick or **1 byte per tick for 10,000 ticks**.
  - This allows the host to send bursty data as long as there are tokens available in the bucket.

Advantage:

- The token bucket algorithm is more flexible than the leaky bucket because it allows bursty traffic, especially if the host has accumulated tokens during idle periods.
- It can adjust to variable traffic patterns while still ensuring that the average data rate does not exceed a specified limit.

Disadvantage:

- If tokens are not accumulated due to extended periods of idleness, the system may still experience congestion when bursty traffic occurs.



Process depicting how token bucket algorithm works

Feature	Token Bucket Algorithm	Leaky Bucket Algorithm
Traffic Handling	Allows for burst traffic by saving tokens for later use.	Smooths traffic by enforcing a constant output rate.
Token/Bucket Concept	Uses tokens to control data transmission.	Uses a leaky bucket to control data transmission.
Data Dropping	Data waits until enough tokens are available to send.	Excess data is discarded if the bucket overflows.
Rate Flexibility	More flexible with bursts, as tokens accumulate during idle time.	More rigid, as it enforces a steady rate with no allowance for bursts.
Use Case	Suitable for bursty traffic and rate limiting where bursts are allowed.	Suitable for steady rate enforcement and traffic shaping.
Implementation Complexity	Generally more complex due to token management.	Generally simpler as it primarily tracks the bucket's level.
Buffering	Data can be buffered until tokens are available to send.	Limited buffering; excess data is dropped if the bucket overflows.